



BEACH SIMULATOR

Game design insertion map

Where each mechanic can be introduced, how modules can communicate, and which boundaries keep the code simple enough for people to work with.

CODE BASE	Local architecture validated on July 21, 2026
DESIGN BASE	Jesse Schell - The Art of Game Design: A Book of Lenses
CORE RULE	Add rules where they belong. Create a new script only when a new and lasting responsibility actually exists.

1. How to use this map

This document is not permission to build any feature anywhere. It is a decision map: first identify the nature of the mechanic, then change the module that already owns that state or rule.

Rule of thumb

- If a feature changes an existing rule, modify the model that already controls that rule.
- If it changes appearance only, modify the corresponding view or visual module.
- If it schedules something for later, use the spawn timeline or the day lifecycle.
- If it crosses two domains, return a small result and route it through the composition point.
- Create another file only when the concept has its own name, state, and a realistic expectation of growth.

Signs of overengineering

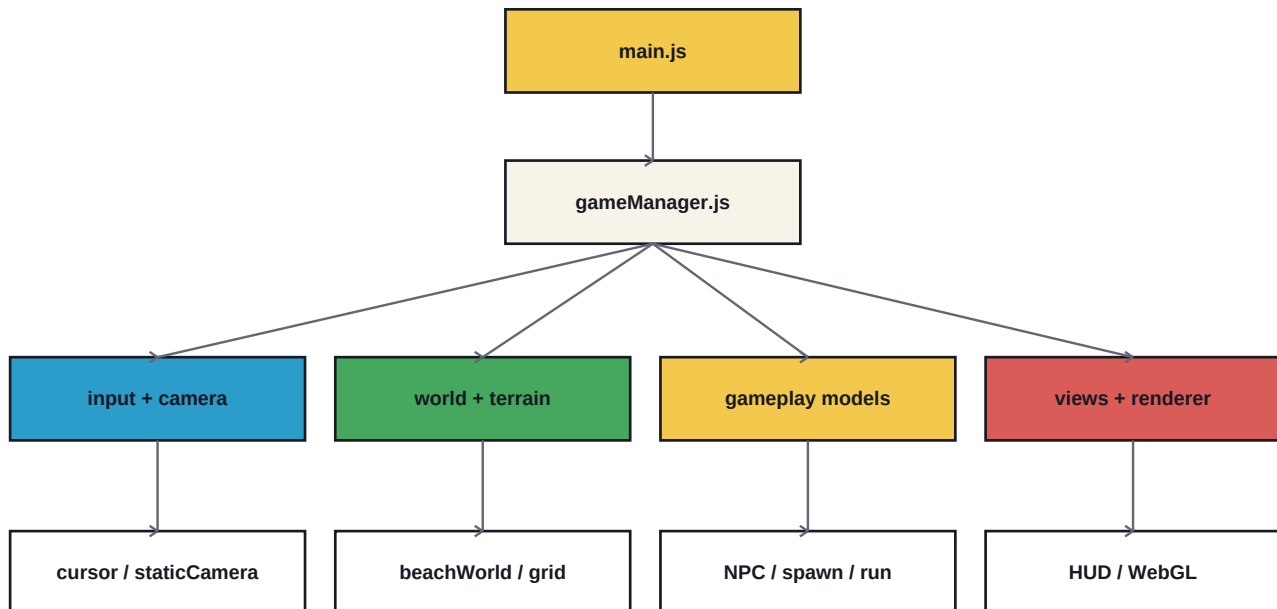
- A new file contains only one call to another file.
- Four scripts must be opened to understand one click.
- A generic Manager knows about camera, UI, NPCs, and money at the same time.
- Events are created for calls that could remain direct and local.
- DTOs, registries, and factories exist without more than one real implementation.

Reading the lenses

The lenses below are used as design questions, not as software patterns. A lens explains why a mechanic may improve the experience; the listed module explains where to implement it without contaminating the architecture.

2. Resulting architecture

The dependency direction remains top-down. **gameManager.js** composes systems and owns the loop. Lower modules never import it. Concrete beach assembly now lives in **world/beachWorld.js**, reducing GameManager without introducing an orchestration framework.



Interpretation

- main.js only starts the game.
- gameManager.js connects input, loop, models, and presentation.
- beachWorld.js assembles terrain, scenery, grid, initial population, and assets.
- Gameplay models do not import the DOM, camera, or renderer.
- Views receive snapshots or presentation commands; they do not decide economy or behavior.

3. Quick map

Use this table to quickly locate the most likely insertion point for a change.

SCRIPT	DESIGN DECISION	EXAMPLE
world/beachWorld.js	Initial map composition	Add a static building, initial position, or reserved area
terrain/beachGrid.js	Playable space and occupancy	Collision, store footprint, blocked cells
terrain/terrainWorld.js	Terrain shape and visuals	Wet sand, vegetation strip, palm density
spawn/spawnManager.js	When something happens	Bather arrival, shark, litter, or treasure
spawn/spawnableObjectDto.js	What may exist	New litter, animal, valuable, or obstacle
npcs/npcSystem.js	What an NPC decides	Needs, conversation, swimming, fleeing, and leaving
buildings/buildingServicesModel.js	Service effect and cost	Items, prices, wages, and maintenance
interaction/cleanBeachController.js	Collection consequence	Money, progress, tools, and combos
onboarding/cleanBeachGuide.js	Contextual first-task help	Delay, target choice, and arrow priority
run/dayLifecycleController.js	Day rhythm	Climax, closing, charges, and spawn opening
camera/staticCamera.js	Camera state and movement	Focus, limits, and minimap data
input/cursor.js	Raw gestures	Touch, drag, zoom, and rotation
ui/*.js	How state is shown	HUD, report, tooltip, notice, and animation

4. World, terrain, and space

world/beachWorld.js	
CURRENT RESPONSIBILITY	Loads assets and assembles the initial beach: terrain, kiosk, beach house, grid, an empty NPC set, and five collection objects.
GAME DESIGN INSERTIONS	• Balance the five initial objects and guaranteed valuable in planInitialPopulation. • Position new buildings that already exist at the start of a run. • Reserve footprints so objects and NPCs cannot appear inside buildings. • Define the initial visual composition that creates the run's first impression.
COMMUNICATION NODE	planInitialPopulation returns DTO requests; loadBeachWorld returns concrete references and resolveWorldObjectPosition to GameManager.
DO NOT PUT HERE	Prices, maintenance, NPC behavior, timers, or reward rules.
BOOK FOUNDATION	Lens #21 Functional Space (p. 135) ; #62 Inherent Interest (p. 254) ; #72 Indirect Control (p. 293) .

terrain/beachGrid.js	
CURRENT RESPONSIBILITY	Divides the map into sea, sand, and vegetation zones; converts positions into cells and tracks occupancy.
GAME DESIGN INSERTIONS	• Building footprints with different dimensions. • An isWalkable query for simple bather collision. • Navigation costs for dry sand, wet sand, and shallow water.
COMMUNICATION NODE	NPC placement and object placement can query the grid through an injected function. The grid must not know specific NPCs or buildings.
DO NOT PUT HERE	Animation, money, timed spawns, or renderer access.
BOOK FOUNDATION	Lens #21 Functional Space (p. 135) ; #22 Dynamic State (p. 140) ; #42 Simplicity/Complexity (p. 196) .

terrain/terrainWorld.js	
CURRENT RESPONSIBILITY	Builds and updates the visual sand, vegetation, sea, and palm cells around the camera.
GAME DESIGN INSERTIONS	• A wet-sand strip or visual variation by zone. • Palm distribution, scale, and spacing. • Purely visual decoration and the terrain's infinite window.
COMMUNICATION NODE	May receive simple visual state, such as tide intensity, without importing TimeManager.
DO NOT PUT HERE	Collision, land price, needs, or spawn rules.
BOOK FOUNDATION	Lens #7 Elemental Tetrad (p. 43) ; #63 Beauty (p. 255) ; #72 Indirect Control (p. 293) .

5. Spawning and objects

spawn/spawnManager.js

CURRENT RESPONSIBILITY	Schedules events, controls arrival intervals, and turns bathers into future litter sources.
GAME DESIGN INSERTIONS	• New channels: shark, crab, jellyfish, customer rush, or treasure. • Probability, cooldown, priority, and attention cost for each event. • Adaptive pacing using rating and, later, a summary of the player's behavioral DNA.
COMMUNICATION NODE	Returns spawn requests. It does not instantiate 3D models or directly change the HUD or economy.
DO NOT PUT HERE	WebGL position, animation, charges, or NPC dialogue.
BOOK FOUNDATION	Lens #2 Surprise (p. 26) ; #29 Chance (p. 169) ; #39 Time (p. 189) ; #61 Interest Curve (p. 252) .

spawn/scheduledSpawnQueue.js

CURRENT RESPONSIBILITY	Maintains the time-ordered queue of future events.
GAME DESIGN INSERTIONS	• Only technical tie-breaking criteria required by scheduling. • Cancellation support when an event's gameplay condition disappears.
COMMUNICATION NODE	Receives event DTOs from SpawnManager and returns the next event. It must remain unaware of beaches, NPCs, or sharks.
DO NOT PUT HERE	Probabilities, narrative, rewards, or event-type selection.
BOOK FOUNDATION	Lens #39 Time (p. 189) ; #43 Elegance (p. 198) .

spawn/spawnableObjectDto.js

CURRENT RESPONSIBILITY	Universal catalog of object types, categories, zones, sources, and traits that may exist in the world.
GAME DESIGN INSERTIONS	• Register a coconut, jewel, rock, log, animal, or new type of litter. • Declare reusable traits such as dirty, pickup, obstacle, or predator. • Define the zones and sources where an object is allowed.
COMMUNICATION NODE	Other modules query the DTO by type. The DTO describes; it does not execute behavior.
DO NOT PUT HERE	Spawn timing, prices, animation, or concrete effects on NPCs.
BOOK FOUNDATION	Lens #22 Dynamic State (p. 140) ; #23 Emergence (p. 143) ; #26 Rules (p. 150) .

6. NPCs and buildings

npcs/npcSystem.js	
CURRENT RESPONSIBILITY	Maintains entities, movement, acceleration, activities, needs, complaints, and departures from the beach.
GAME DESIGN INSERTIONS	• Interaction between two bathers: conversation, friendship, argument, or family group. • Name, age, hygiene profile, heat tolerance, and final review. • States for swimming, fleeing an animal, buying a drink, and seeking a toilet.
COMMUNICATION NODE	update may return small events such as departures and interactions. GameManager routes those results to rating, economy, or UI.
DO NOT PUT HERE	DOM, visual speech bubbles, GLTF loading, money, or global scheduling.
BOOK FOUNDATION	Lens #16 Player (p. 106) ; #19 Needs (p. 127) ; #23 Emergence (p. 143) ; #44 Character (p. 199) .

npcs/npcWorld.js	
CURRENT RESPONSIBILITY	Loads the PicoCAD/GLTF model and converts NPC snapshots into renderable bodies and legs.
GAME DESIGN INSERTIONS	• New animations for swimming, complaining, buying, and talking. • Visual bather variations that preserve the same behavior. • Explicit orientation offsets for each new model.
COMMUNICATION NODE	Receives ready NPC snapshots from NpcSystem. It does not decide where or why an NPC moves.
DO NOT PUT HERE	Needs, pathfinding, ratings, prices, or departure rules.
BOOK FOUNDATION	Lens #7 Elemental Tetrad (p. 43) ; #44 Character (p. 199) ; #57 Feedback (p. 230) .

6. NPCs and buildings - services

buildings/buildingServicesModel.js	
CURRENT RESPONSIBILITY	Tracks owned buildings and calculates their effects, maintenance, and end-of-day costs.
GAME DESIGN INSERTIONS	• Beverage Store items and prices while the catalog remains small. • Stock, margin, lifeguard wages, and toilet maintenance cost. • Clear trade-offs: income versus litter, safety versus wages, hygiene versus maintenance.
COMMUNICATION NODE	Receives commands such as addBuilding or purchase. Returns a financial result; the economy records the transaction.
DO NOT PUT HERE	3D models, choice modal, NPC movement, or temporal event ordering.
BOOK FOUNDATION	Lens #32 Meaningful Choices (p. 181) ; #33 Triangularity (p. 182) ; #46 Economy (p. 204) ; #47 Balance (p. 205) .

scenery/sceneryWorld.js	
CURRENT RESPONSIBILITY	Defines assets, scale, orientation, and visual instances for the kiosk, beach house, and placeholders.
GAME DESIGN INSERTIONS	• Register the visual asset for toilets, trash cans, Wi-Fi spot, and beverage store. • Correct model-specific yaw, size, and brightness.
COMMUNICATION NODE	Receives type and position from world composition or the building choice.
DO NOT PUT HERE	Prices, service effects, maintenance, or random run selection.
BOOK FOUNDATION	Lens #7 Elemental Tetrad (p. 43) ; #63 Beauty (p. 255) .

7. Interaction, input, and camera

interaction/cleanBeachController.js

CURRENT RESPONSIBILITY	Applies collection consequences: money, experience, and task progress.
GAME DESIGN INSERTIONS	• Different values for coins, rings, and jewels. • A tool that removes a specific hazard. • A combo or bonus for rapid cleanup streaks, if playtesting supports it.
COMMUNICATION NODE	Receives the collected object and calls only the involved models. Returns a small result for visual feedback.
DO NOT PUT HERE	Hit testing, DOM, animation, camera, or object creation.
BOOK FOUNDATION	Lens #40 Reward (p. 191) ; #49 Visible Progress (p. 214) ; #57 Feedback (p. 230) ; #58 Juiciness (p. 233) .

interaction/worldObjectInteraction.js

CURRENT RESPONSIBILITY	Projects 3D objects onto the canvas or HUD and resolves hover or click through pixel-based trigger areas.
GAME DESIGN INSERTIONS	• Box selection, building drag, or priority between overlapping objects. • Category-specific hit-area tuning for accessibility. • Positioning new speech bubbles or indicators attached to world entities.
COMMUNICATION NODE	Returns a selection or projected coordinates. A gameplay controller decides the effect of the click.
DO NOT PUT HERE	Money, tasks, inventory removal, or NPC rules.
BOOK FOUNDATION	Lens #48 Accessibility (p. 213) ; #53 Control (p. 222) ; #55 Virtual Interface (p. 226) .

input/cursor.js

CURRENT RESPONSIBILITY	Normalizes mouse and keyboard into pan, rotation, zoom, selection, and edge-scrolling intents.
GAME DESIGN INSERTIONS	• Mobile mapping for pinch, tap, one-finger drag, and two-finger rotation. • Sensitivity and direction-inversion settings.
COMMUNICATION NODE	Emits input intents. It never calls NPC, terrain, or GameManager directly.
DO NOT PUT HERE	Rewards, collision, spawning, concrete camera state, or world mutation.
BOOK FOUNDATION	Lens #35 Head and Hands (p. 185) ; #53 Control (p. 222) ; #54 Physical Interface (p. 226) .

7. Interaction, input, and camera - camera

camera/staticCamera.js	
CURRENT RESPONSIBILITY	Maintains the RTS camera target, distance, yaw, pan, rotation, and zoom.
GAME DESIGN INSERTIONS	• Focus on an NPC, building, or urgent event. • Navigation limits and a smooth return to gameplay. • Provide target and bounds for a future minimap.
COMMUNICATION NODE	For a minimap, the composition point reads camera.getTarget and sends a snapshot to MinimapView. The camera does not import the view.
DO NOT PUT HERE	DOM, minimap icons, day rules, spawning, or money.
BOOK FOUNDATION	Lens #53 Control (p. 222) ; #56 Transparency (p. 227) ; #64 Projection (p. 257) .

8. Run, time, and metrics

run/dayLifecycleController.js	
CURRENT RESPONSIBILITY	Starts the day, enables spawning, ends the day, stops the timeline, and records service costs.
GAME DESIGN INSERTIONS	• Trigger a climax during the last forty seconds. • Prepare the final summary and transition to the next day. • Apply run modifiers before spawning is enabled.
COMMUNICATION NODE	Coordinates day-cycle models, but does not assemble UI or decide individual NPC behavior.
DO NOT PUT HERE	Rendering, camera, assets, dialogue, or a building's internal rules.
BOOK FOUNDATION	Lens #39 Time (p. 189) ; #60 Modes (p. 240) ; #61 Interest Curve (p. 252) .

run/runSessionModel.js	
CURRENT RESPONSIBILITY	Stores the current day, total days, and current run phase.
GAME DESIGN INSERTIONS	• Progression from Day 1 through Day 5. • States for day-intro, active, day-complete, and run-complete. • Run victory or defeat conditions after they are approved.
COMMUNICATION NODE	The lifecycle requests transitions; views observe snapshots routed by the composition point.
DO NOT PUT HERE	Per-frame countdown, UI, charges, or spawning.
BOOK FOUNDATION	Lens #25 Goals (p. 149) ; #39 Time (p. 189) ; #60 Modes (p. 240) .

time/timeManager.js	
CURRENT RESPONSIBILITY	Controls day duration, remaining time, start, and clock updates.
GAME DESIGN INSERTIONS	• Different duration by difficulty or run modifier. • Explicit pause during modals, if the design adopts that rule. • A final-window signal for the day's climax.
COMMUNICATION NODE	Returns a time snapshot. Other modules choose how to react to time.
DO NOT PUT HERE	Concrete spawns, wages, tasks, animation, or clock text.
BOOK FOUNDATION	Lens #18 Flow (p. 122) ; #39 Time (p. 189) ; #61 Interest Curve (p. 252) .

economy/beachEconomyModel.js	
CURRENT RESPONSIBILITY	Maintains the balance and a minimal history of income and expenses in cents.
GAME DESIGN INSERTIONS	• Transaction categories for the end-of-day report. • Cash or debt limits only if they become an approved rule.
COMMUNICATION NODE	Services and controllers send transactions with sourceId. Economy does not ask why the purchase happened.
DO NOT PUT HERE	Item prices, sale probability, UI, or customer behavior.
BOOK FOUNDATION	Lens #5 Endogenous Value (p. 32) ; #40 Reward (p. 191) ; #46 Economy (p. 204) .

8. Run, time, and metrics - reviews

ratings/beachRatingModel.js	
CURRENT RESPONSIBILITY	Stores reviews and calculates the average used by the HUD and spawn frequency.
GAME DESIGN INSERTIONS	• Bather departure review based on cleanliness, fulfilled needs, and safety. • Review weighting or recency after playtesting. • A summary of positive and negative reasons at day close.
COMMUNICATION NODE	NpcSystem returns departures; a controller converts the final experience into a rating and calls submitRating.
DO NOT PUT HERE	Move NPCs, draw stars, directly alter spawning, or decide animation.
BOOK FOUNDATION	Lens #20 Judgment (p. 128) ; #40 Reward (p. 191) ; #57 Feedback (p. 230) .

experience/playerExperienceModel.js	
CURRENT RESPONSIBILITY	Accumulates signals of mastery, pressure, exploration, fatigue, and need for guidance.
GAME DESIGN INSERTIONS	• Adjust only pacing, hints, or future event selection. • Detect a lost player without punishment or hidden reward reduction.
COMMUNICATION NODE	Exposes a compact snapshot to SpawnManager or onboarding. It does not receive DOM or directly control the timeline.
DO NOT PUT HERE	Decide financial results, manipulate ratings, or replace clear rules with invisible difficulty.
BOOK FOUNDATION	Lens #16 Player (p. 106) ; #18 Flow (p. 122) ; #31 Challenge (p. 179) .

9. UI, feedback, and rendering

ui/buildingChoice.js

CURRENT RESPONSIBILITY	Maintains the current roguelike choice and presents three options to the player.
GAME DESIGN INSERTIONS	• Show the benefit, daily cost, and risk of each building. • Ensure the selected consequence appears within the same minute. • Vary options by run without creating one dominant strategy.
COMMUNICATION NODE	The view returns only the selected type. BuildingServices applies the rule; scenery creates the visual.
DO NOT PUT HERE	Charge money, spawn NPCs, or directly alter terrain.
BOOK FOUNDATION	Lens #32 Meaningful Choices (p. 181) ; #33 Triangularity (p. 182) ; #40 Reward (p. 191) .

ui/taskList.js

CURRENT RESPONSIBILITY	Maintains and presents tasks, priority, dependencies, and progress.
GAME DESIGN INSERTIONS	• Concrete short-term goals with a counter and bar. • Conditional tasks for cleanliness, satisfaction, or safety. • Reveal the next task only when the player has enough context.
COMMUNICATION NODE	Controllers advance tasks by id. The view reacts to snapshots and does not query the world.
DO NOT PUT HERE	Remove objects, pay rewards, or decide spawning.
BOOK FOUNDATION	Lens #25 Goals (p. 149) ; #49 Visible Progress (p. 214) ; #57 Feedback (p. 230) .

onboarding/cleanBeachGuide.js + feedback views

CURRENT RESPONSIBILITY	The guide selects a pending item after inactivity; OnboardingView and PickupFeedbackView present the hand, arrow, notices, +1, and \$5.
GAME DESIGN INSERTIONS	• Tune the 3-to-5-second window using playtest results. • Prioritize visible, valuable, or distant objects according to tutorial intent. • Add a second contextual hint without placing the rule inside the view.
COMMUNICATION NODE	CleanBeachGuide queries task and objects, chooses the target, and sends projected coordinates. Views only draw the feedback.
DO NOT PUT HERE	Change money, remove objects, complete tasks, or control the camera.
BOOK FOUNDATION	Lens #48 Accessibility (p. 213) ; #57 Feedback (p. 230) ; #58 Juiciness (p. 233) .

rendering/worldRenderer.js

CURRENT RESPONSIBILITY	Executes WebGL, shaders, sprites, lighting, and the final frame at a fixed internal resolution.
GAME DESIGN INSERTIONS	• PSX shader, jittering, water, transparency, fog, and graphics optimization. • New visual passes only when an effect cannot fit the current material.
COMMUNICATION NODE	Receives ready sceneObjects. It does not query GameManager or domain models.
DO NOT PUT HERE	Ratings, spawning, selection, prices, collision, or NPC behavior.
BOOK FOUNDATION	Lens #7 Elemental Tetrad (p. 43) ; #63 Beauty (p. 255) ; #64 Projection (p. 257) .

10. Three insertion examples

A. Store items and prices

- buildings/buildingServicesModel.js: a small list of beverages, prices, margins, and stock.
- npcs/npcSystem.js: BUYING state and the decision to seek the store.
- economy/beachEconomyModel.js: records only the already-calculated sale.
- scenery/sceneryWorld.js: store asset and visual orientation.
- ui/buildingChoice.js: communicates the trade-off before selection.

B. Two bathers interact

- npcs/npcSystem.js detects proximity and creates an interaction result.
- The same system changes both NPC states to TALKING or ARGUING.
- npcs/npcWorld.js animates both bodies.
- A view receives dialogue only when text actually exists.
- SpawnManager does not participate: the interaction comes from current state, not a global clock.

C. Opening a minimap communication node

- camera/staticCamera.js already provides target and orientation.
- terrain/beachGrid.js provides zones and logical boundaries.
- NPCSystem and world objects provide minimal position snapshots.
- A future MinimapView receives one simple object containing those values.
- GameManager only assembles the snapshot during render; camera, NPC, and grid do not import the view.

Suggested contract, only after the minimap is approved

```
minimapView.render({ cameraTarget, cameraYaw, zones, npcs, objects })
```

11. Recommended backlog

Suggested order for improving the experience without concentrating rules in GameManager again.

SCRIPT	DESIGN DECISION	EXAMPLE
1. Five days	Close the core loop	advanceDay, clock reset, and run-complete
2. NPC collision	Preserve spatial readability	Inject beachGrid.isWalkable into NpcSystem
3. Meaningful buildings	Make choices matter	Visible trade-offs and consequences within the same minute
4. First customer	Create attachment and curiosity	Name, need, and first review
5. Complete rating loop	Close systemic feedback	Departure creates a review and changes future frequency
6. Climax and report	Shape the five minutes	Final event, balance, rating, cleanup, and teaser
7. Emergent events	Increase variety	NPC interactions, animals, litter, and treasure

Criterion for creating a new script

Create a file only when at least two conditions are true: it has its own state; it has its own rule; it has more than one consumer; the concept deserves its own tests; the current file would become difficult to understand with the feature. Otherwise, modify the existing method or module.

Priority lenses for the next five minutes

- #25 Goals: the player always knows what to do now.
- #32 Meaningful Choices: every building changes the run.
- #39 Time: events fit the five-minute rhythm.
- #49 Visible Progress: cleanliness and satisfaction have concrete evidence.
- #57 Feedback and #58 Juiciness: every action responds immediately.
- #61 Interest Curve: hook, rest, escalation, climax, and resolution.

Atualizacao de Game Design - Economia

Inspiracao: Factorio Learning Environment. O primeiro incremento aplica a ideia de Production Score como uma leitura continua da atividade economica, sem transformar a economia em estado global.

Implementado

economy/beachEconomyModel.js agora expoe receita acumulada, despesas acumuladas, variacao liquida e atividade economica acumulada. A atividade e a soma absoluta de entradas e saidas e serve como uma medida de throughput financeiro da run.

Efeito no jogo

O saldo continua sendo a fonte autoritativa para compras e manutencao. As novas metricas sao somente observaveis neste ciclo e permitem, em um proximo passo, exibir desempenho e criar marcos sem duplicar regras de dinheiro em UI ou gameplay.

Atualizacao visual

O botao de construcao deixou de depender do texto BUILDING e agora usa o asset creft-thumb.png como thumbnail. O comportamento de clique, bloqueio por saldo e os estados visuais continuam no mesmo componente.

Legibilidade dos textos

Os fundos dos overlays e contadores textuais foram removidos: status, reclamacoes, feedback de coleta, relógio, tarefas e FPS. Fundos de botoes, dialogos e barras de progresso permanecem como elementos de interacao e estrutura visual.

Contador de banhistas

O texto numerico de bather-counter__count passou de 1.6rem para 3.2rem e recebeu rotacao visual de 30 graus. O thumbnail e os demais contadores nao foram alterados.

Proximo incremento sugerido

Adicionar marcos de progressao baseados em operacoes reais da praia, com regras explicitas e limite por run. Esse passo deve ser separado da camada de apresentacao para preservar a direcao gameManager -> modelos -> UI.